

CSS233 WEB PROGRAMMING I

LECTURE 1 WEB FUNDAMENTALS

Dr. Wittawin Susutti

wittawin.sus@kmutt.ac.th

ดร.วิธวินท์ สุธุทธิ (อ.แกส)

wittawin.sus@kmutt.ac.th

อาคารปฏิบัติการพื้นฐานทางวิทยาศาสตร์ ชั้น 7 SCL705

เวลาให้คำปรึกษา พุธ 9.00-12.00 น.

Objectives

- Write well-structured, easily maintained, W3C standards-compliant, **markup languages, style sheets**.
- Write client and server-side scripting languages such as **JavaScript** and **CSS**.
- Use JavaScript to access and use **web services** for dynamic content (AJAX, JSON, etc.).
- Develop **databases** using SQL and NoSQL.

Lesson plan

Week	Topic
1	Web Fundamentals
2	Semantic HTML
3	CSS Layout
4	JavaScript
5	TypeScript
6	Next.js Routing
7	Lab test 1
8	Backend API
9	Web–Database Integration
10	Project Progress
Exam 1	
11	Authentication
12	Lab test 2
13	Testing Fundamentals
14	Deployment
15	Project presentation
Exam 2	

Assessment & Textbooks

วิธีการวัดและประเมินผล	สัดส่วนของการวัดและ การประเมินผลรวม
งานภาคปฏิบัติ	20%
Project	30%
Lab test	30%
Midterm exam	20%

Textbook:

- David Flanagan, JavaScript: The Definitive Guide 7th Edition, O'Reilly Media, Inc., 2020.
- Marijn Haverbeke, Eloquent JavaScript 3rd edition, No Starch Press, 2018.
- Cory Lindley, Front-End Developer Handbook, Frontend Masters, 2018.

AI use is allowed—but it must be disclosed, verified, and explained

Allowed

- Ask for explanations
- Get debugging help
- Request tests/checklists
- Review work

Required

- Keep an AI Use Log
- Verify against evidence
- Record accepted/rejected ideas
- Explain the result

Prohibited

- Submit work you do not understand
- Hide AI use
- Share secrets or personal data

The Internet and The Web

Internet

The global system of interconnected computer networks that use the Internet protocol suite (TCP/IP) to link devices worldwide.

Web

Linked resources accessed with Uniform Resource Locators (URLs) and HyperText Transfer Protocol (HTTP)

The Internet supports other services such as email.

The Web is one service built on the Internet.

The Internet vs World Wide Web

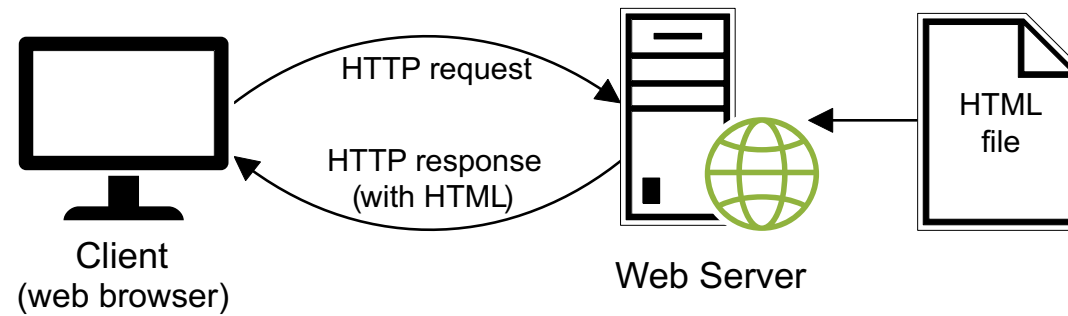
The Internet	World Wide Web
Definition	
A global system of interconnected computer networks that use the TCP/IP protocol to link devices worldwide	Online content that is formatted in HTML and accessed via HTTP protocol.
Functionality	
A massive interconnection of computer networks around the world.	Service provided by the Internet.
Protocol	
Transmission Control Protocol/Internet Protocol (TCP/IP)	Hyper Text Transfer Protocol (HTTP)

Static vs Dynamic web

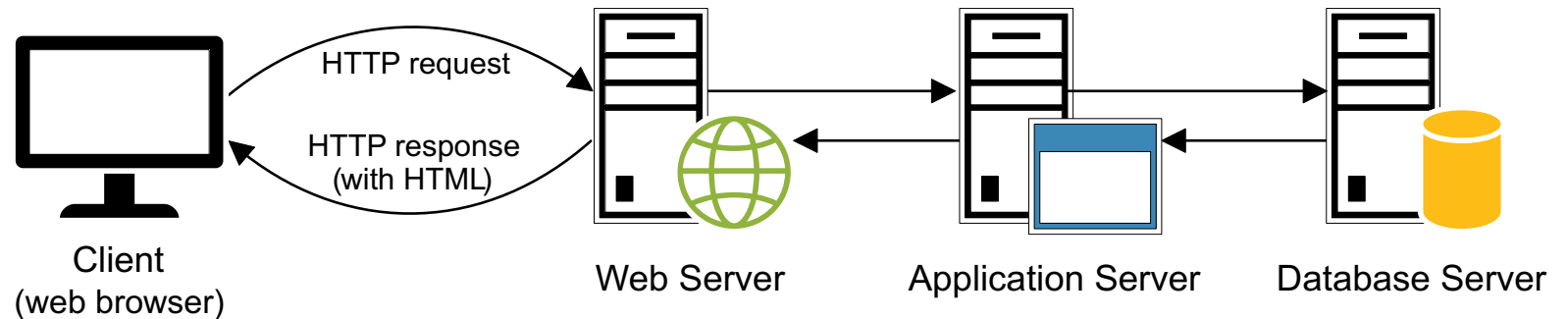
- Static web page (flat page/stationary page) is a web page that is delivered to the user exactly as stored.
 - displays the same information for all users, from all contexts
- Dynamic web pages generated by a **web application**.
 - A **server-side dynamic web page** - construction is controlled by an application server processing server-side scripts.
 - A **client-side dynamic web page** processes the web page using scripting running in the browser as it loads.

How a web server processes a web page

Static



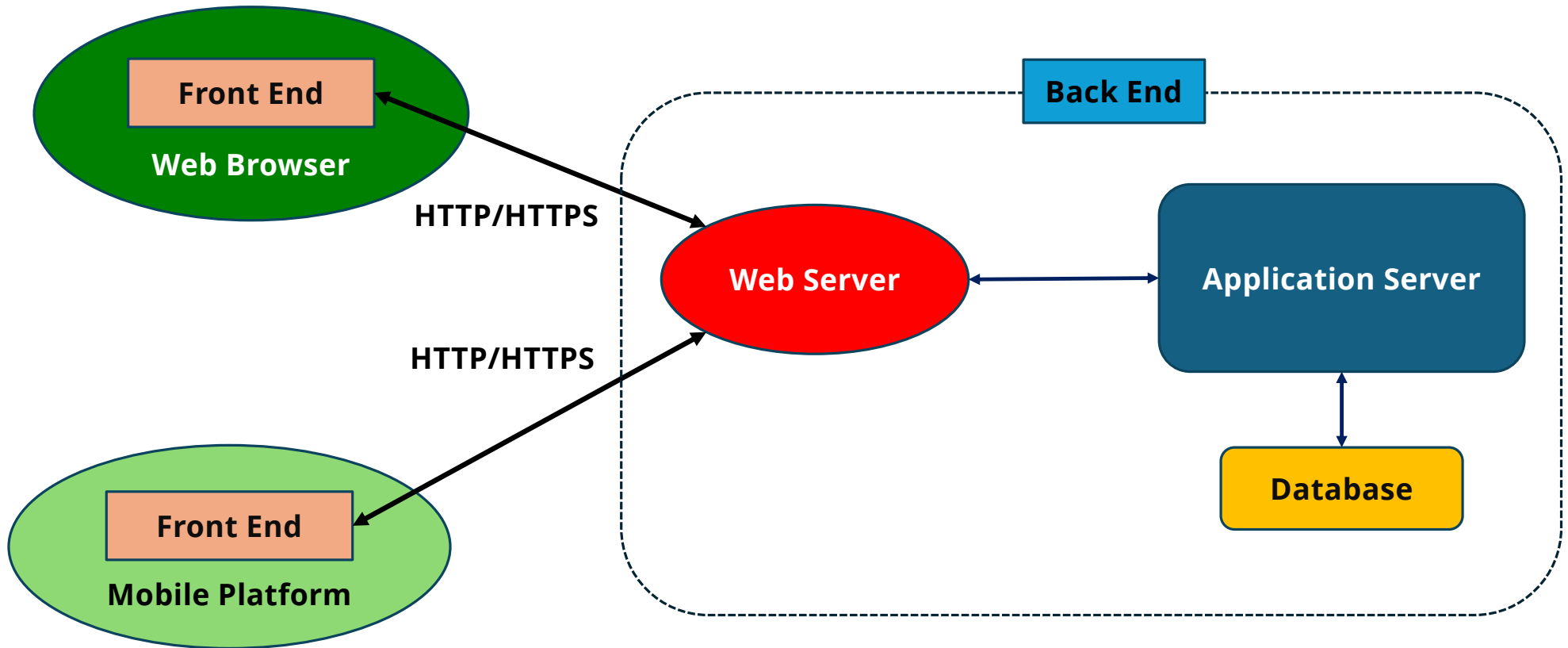
Dynamic



What is a Web Application?

- A program that the user interacts with through the Internet.
- Five components
 - HTTP Server
 - HTTP Client (or Web Browser)
 - Database
 - Client-Side Programs (Front-end)
 - Server-Side Programs (Back-end)

Web Application Architecture





Front-end vs Back-end

- The front-end is the stuff that you see and interact with: HTML, CSS, Javascript.
- The back-end is everything else.
- Restaurant analogy
 - Front-end is what is plated and sent to your table.
 - Back-end is everything that happens in the kitchen.

<https://roadmap.sh/>

One keystroke triggers multiple systems

```
https://example.com/notices?id=42
```

- How does the browser know where to connect?
- What does the request ask for, and what does the response return?
- Where can we find evidence when a URL is wrong?
- How does our code become a page the user can see?

Clients request; servers respond

User

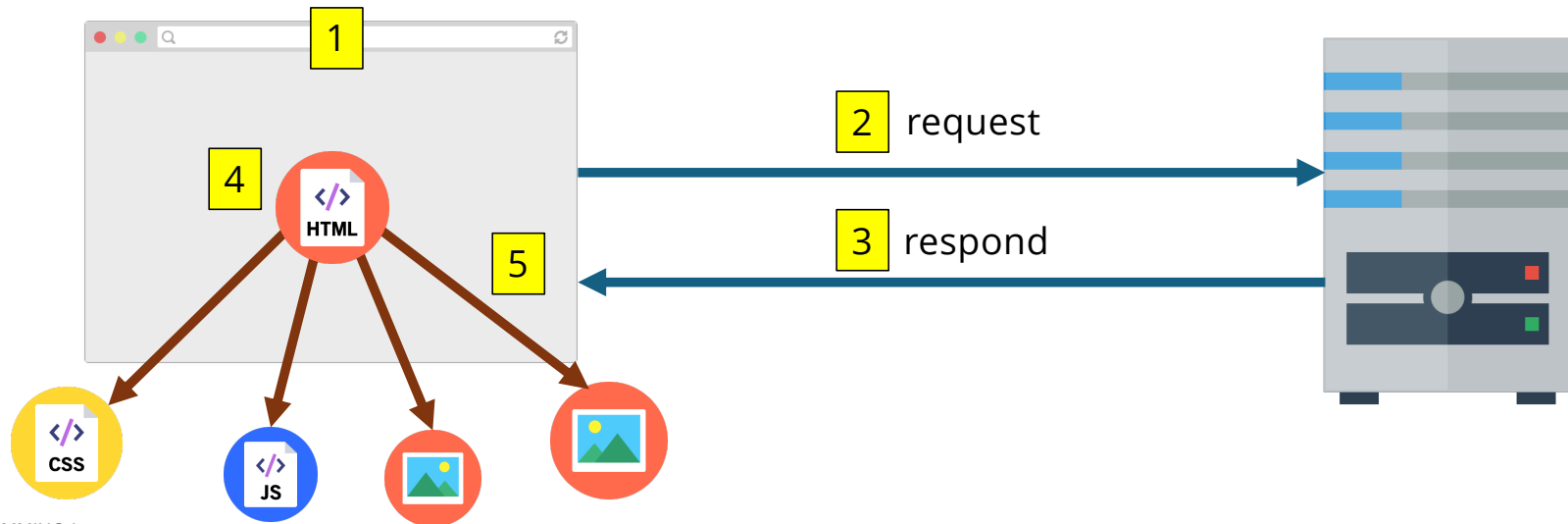
States a goal, such as opening the notices page

Client / Browser

Creates an HTTP request, receives the response, and renders content

Server

Receives the request, processes it, and returns status, headers, and body



A URL identifies access method, location, and resource details

```
https://example.com:443/notices?id=42#details
```

Scheme + Host

https selects the protocol
example.com identifies the host

Port + Path

443 selects the port
/notices identifies the resource path

Query + Fragment

?id=42 is sent to the server
#details stays in the client

- Any character in a URL other than A-Z, a-z, 0-9, or any of -_~ must be represented as %xx, where xx is the hexadecimal value of the character:

```
http://www.stats.com/companyInfo?name=C%26H%20Sugar
```

HTTP message

Request

POST / HTTP/1.1

Host: developer.mozilla.org

User-Agent: curl/8.6.0

Accept: */*

Content-Type: application/json

Content-Length: 345

```
{  
  "data": "ABC123"  
}
```

← Start line →

← Headers →

← Empty line →

← Body →

Response

HTTP/1.1 403 Forbidden

Server: Apache

Date: Fri, 21 Jun 2024 12:52:39 GMT

Content-Length: 678

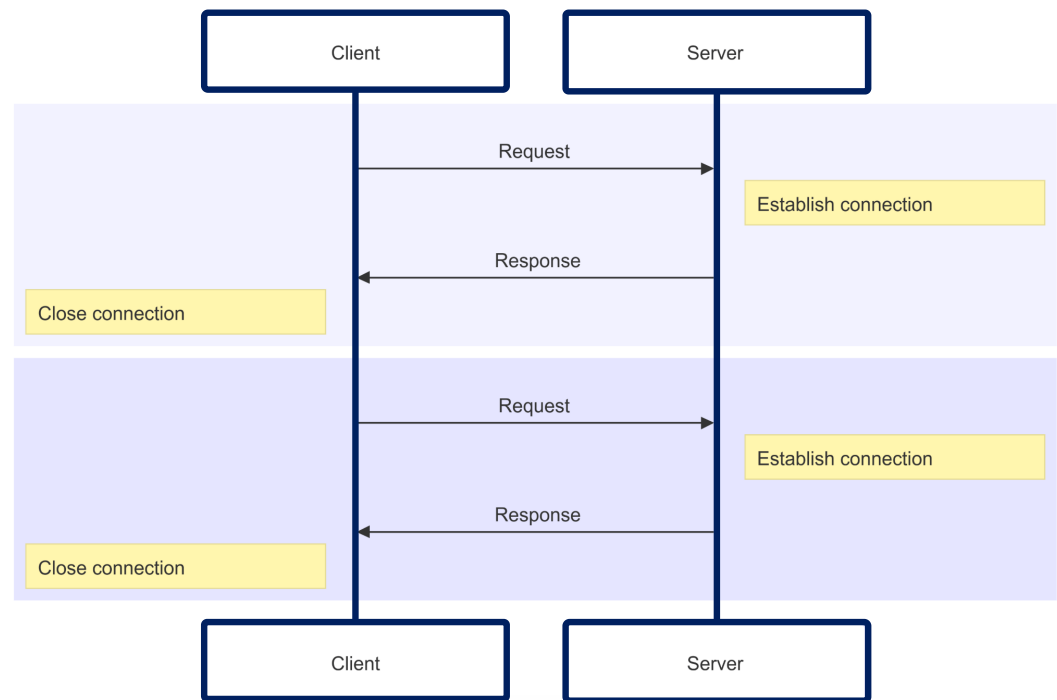
Content-Type: text/html

Cache-Control: no-store

```
<!DOCTYPE html>  
<html lang="en">  
(more data...)
```

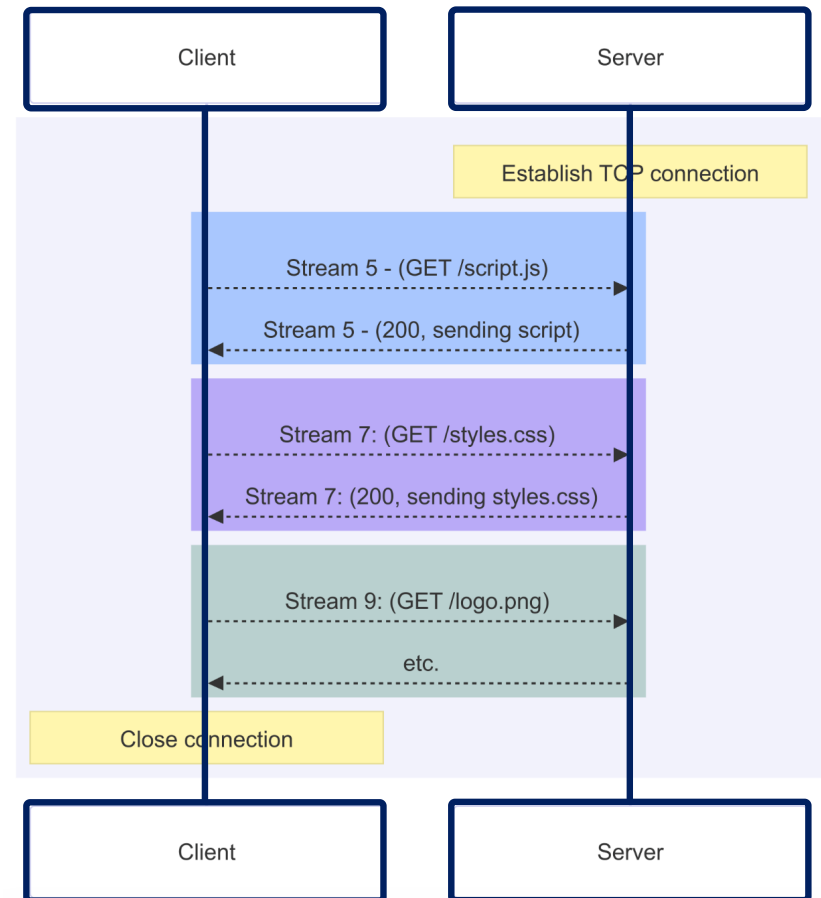
HTTP/1.x

- **Text-based protocol**
- **One request** → **one response** per transaction
- **Request and response are sent sequentially**
- **Headers are plain text**, repeated in every request
- **No header compression**
- **HTTP/1.0**: typically opens a new TCP connection for each request
- **HTTP/1.1**: supports **persistent (keep-alive)** **connections**
- **Pipelining** exists in HTTP/1.1 but is rarely used due to head-of-line blocking
- **Head-of-line (HOL) blocking** limits performance
- Multiple resources often require **multiple TCP connections**
- Simpler implementation but less efficient for modern web applications



HTTP/2

- **Binary protocol** (not human-readable on the wire)
- **Multiplexing**: multiple requests and responses share **one TCP connection**
- **Frames** are the smallest communication unit
- Related frames are grouped into **Streams**
- Multiple streams can be transmitted **concurrently**
- **HPACK header compression** reduces redundant header data
- **Stream prioritization** allows important resources to be delivered first
- **Server Push** (optional, now rarely used by browsers)
- Lower latency and better bandwidth utilization
- Designed to improve performance for websites with many assets (CSS, JS, images, fonts)



Methods carry meaning

GET

Request a representation
Read data

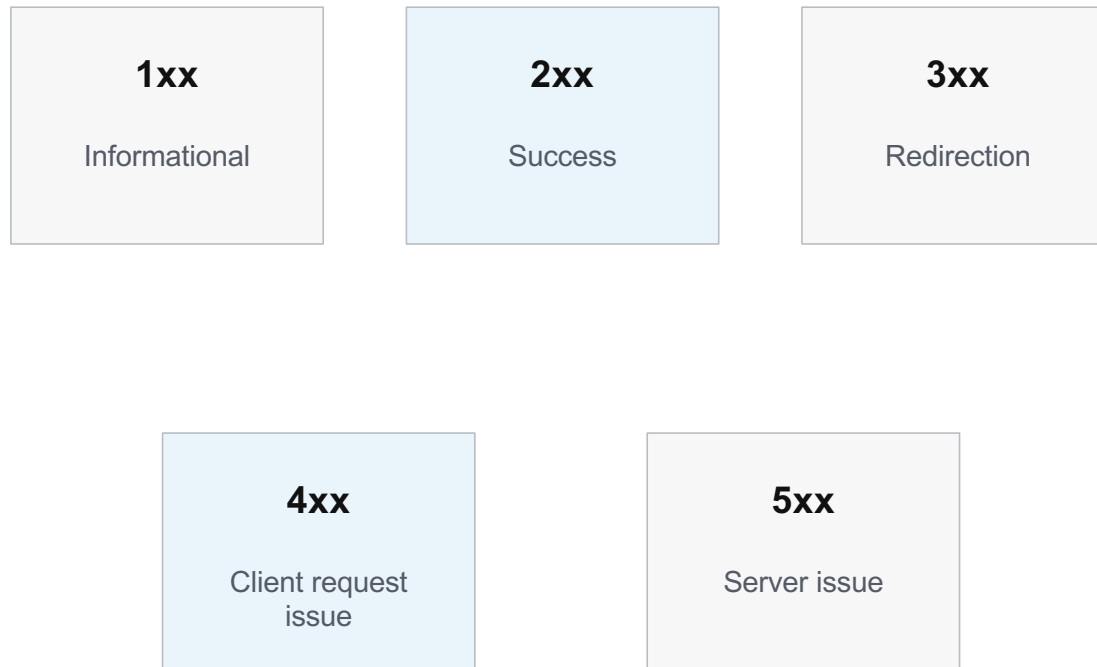
POST

Submit data for processing
May create or change state

PUT / PATCH / DELETE

Replace / partially update /
delete
Explored in Session 8

The first status-code digit identifies the result class



HTTP Status Codes

2xx Success (The "Everything is Fine" Codes)

- These codes indicate that the client's request was successfully received, understood, and accepted.
- **200 OK**
 - The standard response for successful HTTP requests.
 - Used for GET (returning data), PUT (updating data), or DELETE (if returning the deleted object).
- **201 Created**
 - Specifically for POST requests.
 - It signals that a new resource has been successfully created.
- **204 No Content**
 - The request was successful, but there is no data to send back.
 - This is common for DELETE requests or PUT updates where the client doesn't need to see the object again.

HTTP Status Codes

4xx Client Errors (The "You Did Something Wrong" Codes)

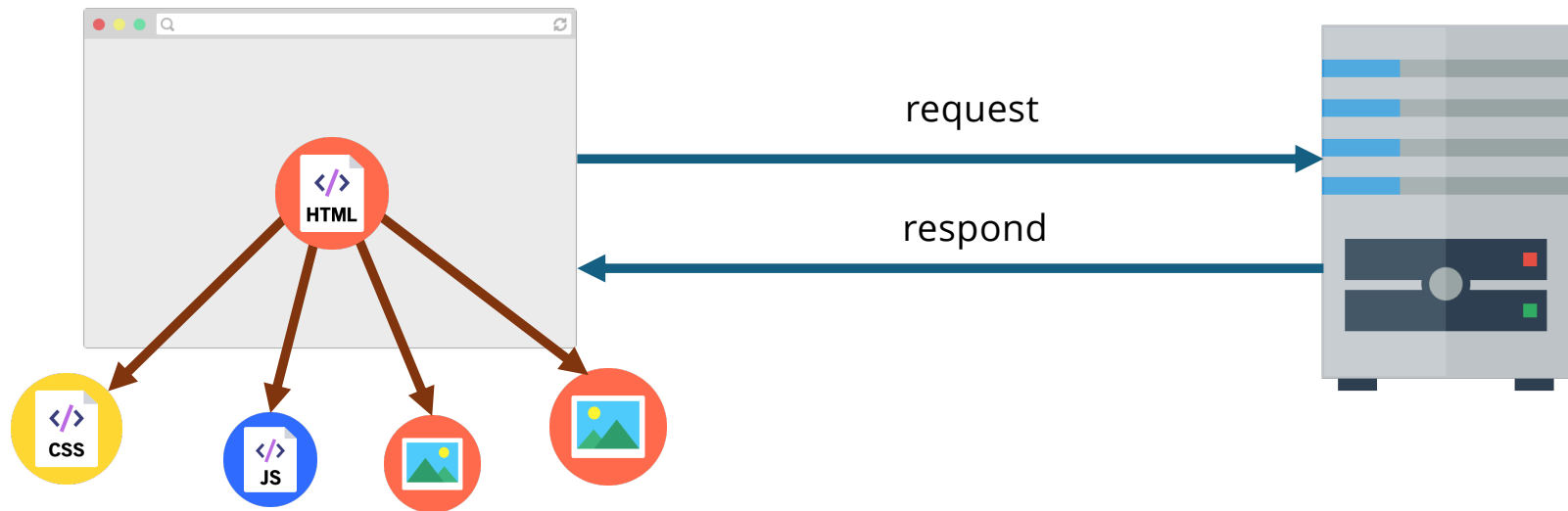
- **400 Bad Request**
 - The server cannot process the request due to client error.
- **401 Unauthorized**
 - The user is not authenticated (need to log in or provide a valid API key/Token).
- **403 Forbidden**
 - The user is authenticated but **does not have permission** to access the specific resource.
- **404 Not Found**
 - The requested resource could not be found.
 - This is frequently used when a specific ID in the URL does not exist in the database.
- **405 Method Not Allowed**
 - The endpoint exists, but the HTTP method used is not supported.
- **429 Too Many Requests**
 - The user has sent too many requests in a given amount of time.

HTTP Status Codes

5xx Server Errors (The "I'm Broken" Codes)

- **500 Internal Server Error**
 - A generic error message when the server encounters an unexpected condition.
- **502 Bad Gateway**
 - The server, while acting as a gateway or proxy, received an invalid response from an upstream server.
- **503 Service Unavailable**
 - The server is currently unable to handle the request due to maintenance or being overloaded.
- **504 Gateway Timeout**
 - The server was acting as a gateway and timed out waiting for a response from the backend.

One page usually requires multiple requests



The browser assembles the page

Each resource has its own URL, status, headers, and body.

Why do web programming?

Hassle-free

- No installation
- Automatic software update
- 24/7 availability

It uses industry standards

- Some web programming components have well-established standards, e.g.: HTTP, HTML5, CSS3, JS

Many of them are free

Drawbacks of web application

Your web application is *generally* slower

- Why?

The offline counterparts are still stronger

- Example?

Drawbacks of web application

Web-apps are more prone to various security attacks

- DoS, SQL injection, Clickjacking, stolen passwords

Harder to guard intellectual property

- It is easy to look at the "source code" of the (client-side) web applications

What is version control?



thesis.docx
thesis-final.docx
thesis-final-v2.docx
thesis-final-v2-REALLY-FINAL.docx
thesis-FINAL-dont-touch.docx

What is version control?

- **Version control** - a system that records changes to files over time so you can recall any specific version later — a *time machine for your files*.
- Older systems stored history as a *diff* — only "what changed" — then replayed the changes to reconstruct a file.
- Git history is a **network of snapshots**, each one linking back to the one before.
 - There are no "lost points."
 - You can jump to any moment in the past, fast and accurately.

Why distributed?

- Git is a *distributed* version control system — everyone on the team holds a **full copy of the entire history** on their own machine.
- Why this matters:
 - **Works offline** — you can commit with no internet.
 - **No single point of failure** — if GitHub goes down, every clone still has the full history.
 - **Experiment fearlessly** — every commit is a point you can return to.

Get git installed

Step 1 — Check if git is already installed

```
git --version
```

Step 2 — Install it (if not present)

Then run `git --version` again to confirm.

Step 3 — Tell git who you are

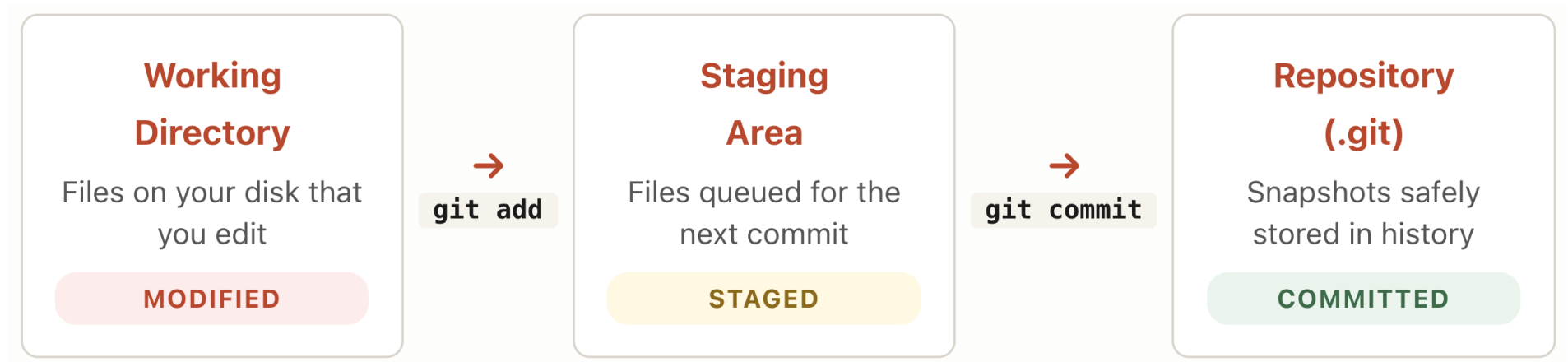
```
git config --global user.name "Your Full Name"  
git config --global user.email "you@example.com"
```

Step 4 — Verify the setup

```
git config --global user.name && git config --global user.email
```

The Three States

- When you run git commands, you are *moving a file between these states*.



Make your first commit

Step 1 — Create a folder and turn it into a repository

```
mkdir my-first-repo  
cd my-first-repo  
git init
```

Step 2 — Create a file and check its state

```
create README.md and type "Hello git!"  
git status
```

Step 3 — Stage the file

```
git add README.md  
git status
```

Step 4 — Commit it

```
git commit -m "Add README with greeting"
```

Step 5 — See your history

```
git log
```

The commit message matters more than you think

- Each commit message is a note to your future self and your team.
- A good message answers "*why does this change exist?*" — not "*what did I change?*".

Weak	Better
fix	Fix login crash when email is empty
update	Add validation for thesis title length
asdf	Remove unused import in parser.py

Remotes

- **Local repository** is a folder on your laptop.
- A **remote** is a **second copy** of that repository living on another server.
- The two stay in sync by sending commits to each other.

Local repo On your laptop
(the `.git/` folder you made with `git init`)

`git push →`

`← git pull`

Remote repo On GitHub's server
(the default remote is named `origin`)

push my-first-repo to GitHub

Step 1 — Create a GitHub account

Step 2 — Set up authentication

Step 3 — Create an empty repo on GitHub

Step 4 — Link your local repo and push

```
git remote add origin <your https on github repo>  
git branch -M main  
git push -u origin main
```

The four commands that sync the world

Command	Direction	What it does
<code>git push</code>	local → remote	Upload your new commits to GitHub
<code>git pull</code>	remote → local	Download new commits <i>and</i> merge them into your branch
<code>git fetch</code>	remote → local	Download new commits <i>without</i> merging — lets you inspect first
<code>git clone <url></code>	remote → (new) local	Download a full copy of a repo you don't have yet

*** **PULL = FETCH + MERGE** ***

Cloning

- If a repo already exists on GitHub (say, a teammate's project, or an open-source library), you don't init — you clone:

```
git clone git@github.com:<username>/<repo>.git
```

- This downloads the *entire history* and automatically sets up origin for you.

Branching

- A branch is a **tiny movable pointer** to one commit.
- Creating a branch costs almost nothing — git just writes down a new pointer.
- The git way is to **branch often**.
- Every experiment, every feature, every bug fix gets its own short-lived branch.

Commits as a graph

```
C1 ← C2 ← C3    main ← HEAD
```

```
git switch -c add-author
```

```
C1 ← C2 ← C3    main
                  |
                  └─ add-author ← HEAD
```

```
C1 ← C2 ← C3    main
                  |
                  └─ C4 ← C5    add-author ← HEAD
```

Branch-edit-merge

Step 1 — See where you are

```
git status  
git branch
```

Step 2 — Create a branch and switch to it

```
git switch -c add-author  
git branch
```

Step 3 — Make a change on the branch

```
git status  
git add README.md  
git commit -m "Add author line to README"
```

Step 4 — Switch back to main and confirm the change isn't there

```
git switch main  
cat README.md
```

Step 5 — Merge the branch into main

```
git switch main  
git merge add-author  
git log --oneline --graph
```

Cleaning up: delete a merged branch

```
git branch -d add-author
```

What a conflict actually is

- When two branches both have new commits, git tries to combine them.
- For most changes this is easy — branch A touched file X, branch B touched file Y, so git just takes both.
- A **merge conflict** happens in exactly one situation: **both branches changed the same lines of the same file, in different ways.**
- A conflict is not an error, not a crash, and not damage.
- git has simply paused mid-merge and is asking: "both versions can't be true at once — which one (or what combination) do you want?", just answer and git continue.

Trigger the conflict

Step 1 — Set up: create new branch

```
git switch -c rename-experiment
```

Step 2 — Commit on main

```
git switch main  
add "# My Amazing Project" in the first line  
git add README.md  
git commit -m "Rename project to Amazing on main"
```

Step 3 — Branch off and edit the *same line* differently

```
git switch rename-experiment  
add "# My Wonderful Project" in the first line  
git add README.md  
git commit -m "Rename project to Wonderful on branch"
```

Step 4 — Merge and watch the conflict appear

```
git switch main  
git merge rename-experiment
```

Step 5 — Let git status guide you

```
git status
```

Trigger the conflict

Step 6 — Open the file and see the markers

```
<<<<<<< HEAD
# My Amazing Project
=====
# My Wonderful Project
>>>>>>> rename-experiment
```

Step 7 — Resolve by hand

Delete all conflict except "# My Amazing Project"

Step 8 — Stage and commit to finish the merge

```
git add README.md
git commit -m "Resolved merge conflict"
git log --oneline --graph -5
```

Pull request

- Everything you've learned so far is part of **git itself**.
- A **Pull Request** (PR) is **not** a git command, it's a feature of *GitHub* (GitLab calls the same thing a "Merge Request").
- A PR is a conversation page on GitHub that wraps around a branch, asking: *"here's my work on this branch — review it, discuss it, and when we're happy, merge it."*
- Teams use PRs because the *conversation* — review, comments, approval — is where quality happens.

Open your first PR

1

Branch

`git switch -c`

2

Commit

`git commit`

3

Push

`git push -u`

4

Open PR

on GitHub

5

Review & merge

on GitHub

Step 1 — Make sure main is up to date

```
git switch main
git pull
```

Step 2 — Branch, edit, commit

```
git switch -c add-license
create LICENSE.md and type "# MIT"
git add LICENSE
git commit -m "Add MIT license placeholder"
```

Step 3 — Push the branch to GitHub

```
git push -u origin add-license
```

Open your first PR

Step 4 — Open the PR on GitHub

- Go to your repo on github.com
- **Pull requests** tab → **Compare & pull request**

Step 5 — Write a meaningful PR description

Step 6 — Click "Create pull request"

Your PR is now open.

GitHub shows:

- The list of commits on the branch
- The **diff** — every line changed, green for added, red for removed
- A **comments** thread for discussion
- At the bottom: a big **Merge pull request** button

Step 7 — Merge the PR

Click **Merge pull request** → **Confirm merge**.

Sample PR description

Title: Add MIT license

What

Adds an MIT LICENSE file to the repository root.

Why

The repo currently has no license, which means the code is "All rights reserved" by default. MIT lets others use it freely.

Testing

Verified the file appears in the repo file listing.

Step 8 — Pull the merged result back to your local

```
git switch main
git pull
git branch -d add-license
```

Code review - the real point of a PR

- In a team, the workflow is
 - You open a PR and **request review** from a teammate
 - They read the diff, leave comments on specific lines, ask questions
 - You reply, push more commits to the same branch (the PR updates automatically)
 - When they're satisfied, they click **Approve**
 - Only then does the PR get merged
- Review catches bugs, spreads knowledge, and enforces the team's quality bar.
- Many teams protect main with a **branch protection rule** that *forbids* merging without an approval — you literally cannot click merge until someone approves.

A kind review culture

- Review comments should critique the *code*, never the *person*.
- "Could this loop be a bug if X is null?" beats "You forgot to handle null."
- When you're reviewed, assume good intent; when you review, be specific and kind.
- This is a skill as important as any git command.

Fork

- To contribute to a repo you *don't* control (open source, a teammate's project), the workflow adds one step: a **fork**.

Step	Your repo	Someone else's repo
0	—	Fork it on GitHub (makes your personal copy)
1	<code>git switch -c</code>	<code>git switch -c</code>
2	<code>git commit</code>	<code>git commit</code>
3	<code>push to origin</code>	<code>push to origin</code> (your fork)
4	open PR: your branch → your main	open PR: your fork → their main (an "upstream" PR)

The single rule that prevents 90% of disasters

Has this commit been pushed to a shared remote, where a teammate might have pulled it?

- **No** (it's local-only, or on a branch only you use) → you can *rewrite* history freely. Use `reset`, `commit --amend`.
- **Yes** (it's on main, or pushed to a PR others can see) → *never rewrite*. Use `revert`, which adds a new commit that undoes the old one.

Choose your undo by "where the change is"

Where is the change you want to undo?

- |
- |— Just edited in the working directory (not staged, not committed)
 - | └─ `git restore <file>` → throw away the edits
- |
- |— Staged with `git add` (not committed yet)
 - | └─ `git restore --staged <file>` → back to working dir
- |
- |— Committed, but NOT pushed
 - | └─ fix the last commit → `git commit --amend`
 - | └─ undo the last commit softly → `git reset --soft HEAD~1`
 - | └─ wipe the last commit fully → `git reset --hard HEAD~1` (destroys work)
- |
- |— Committed AND pushed (shared history)
 - | └─ `git revert <commit>` → adds an "undo" commit, safe for everyone

The commands in detail

git restore — discard uncommitted edits

```
git restore broken-file.txt
```

git commit -amend — fix the last commit

```
# Fix the message of the last commit
```

```
git commit --amend -m "Better message"
```

```
# Add a forgotten file to the last commit
```

```
git add forgotten-file.txt
```

```
git commit --amend --no-edit
```

```
# --no-edit keeps the original message
```

git revert — the safe undo for shared history

```
git revert <commit-hash>
```

On shared branches, **always revert, never reset.**

The commands in detail

git reset — move the branch pointer backward

```
git reset HEAD~1
```

Mode	Staging area	Working directory	Use when
--soft	kept	kept	You want to re-commit the changes differently (e.g. split one commit into two)
--mixed (default)	cleared	kept	You want to un-commit but keep the changes as unstaged edits
--hard	cleared	wiped	You want to throw <i>everything</i> away and start fresh from an older commit

git reset --hard discards uncommitted work *and* rewinds history.

It cannot be undone for the *uncommitted* parts that work was never in a commit, so it's not in the reflog either. Triple-check before pressing enter.

The commands in detail

git revert — the safe undo for shared history

```
git revert <commit-hash>
```

On shared branches, **always revert, never reset.**

git reflog — the ultimate safety net

- Every time HEAD moves — every commit, every checkout, every reset, every merge — git records it in a log called the **reflog**.
- Even after a reset --hard, the "lost" commits are still in the reflog for around 90 days.

Lose something and get it back

Step 1 — Make a commit you'll "lose"

```
git switch main  
create precious.txt type "important work"  
git add precious.txt  
git commit -m "Add precious work"
```

Step 2 — "Accidentally" wipe it with reset --hard

```
git reset --hard HEAD~1  
precious.txt is gone!
```

Step 3 — Find it in the reflog

```
git reflog
```

Step 4 — Restore it

```
git reset --hard <hash-from-reflog>  
precious.txt is back!
```